TOPIC 2

APPROACH IN PROBLEM SOLVING

IPO Charts & Control Structures

COMPUTER SCIENCE II| SC025

Prepared by: Ms Jasini



Learning Outcome

At the end of the lesson, students should be able to:

a) Describe steps in problem solving: **problem analysis**, **design a solution**, **implementation**, **testing** and **documentation**.

b) Identify **input**, **process**, and **output** based on the given problem statement.

Input, Process, Output Chart

RECAP: ***Problem analysis*** ***is the process of*** ***identifying input***, ***process***, ***and*** ***output*** ***for a given*** ***problem statement***.

- To **identify** the **input, process, and output** for a problem, we construct an **IPO (Input Process Output) chart**.
- The **focus** is on the **input, process,** and **output** from the **perspective (POV)** of the **algorithm** or **program** that can be used to solve the problem statement given.

NOTE: This is DIFFERENT from Chapter 1 of SC015 as we are NOT looking at an entire system.

Input, Process, Output Chart

INPUT

- Use ***descriptive names*** to represent input as we do not know what value may be entered by the user:
  - Eg: num1, num2, num3, number, ...
  - Eg: height, weight, price, discountRate, num_students, ...
- **MUST write ALL inputs**.

OUTPUT

- Use ***descriptive names*** for ***value outputs*** whose values are **unknown**.
  - Eg: totalPrice, average_weight, ...
- Use ***specific values*** for ***outputs*** that are **known** and **messages**.
  - Eg: 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, ...
  - Eg: “Hello World!”, “PASS” or “FAIL”, ...
- **ALL possible outputs must be written.**

Input, Process, Output Chart

PROCESS

- Use **appropriate keywords** based on the control structure.
- Keyword Examples:
  - ***calculate*** - when some value is being calculated
  - ***determine*** - when a condition must be evaluated
  - ***repeat / loop*** - when a process needs to be repeated a certain amount of times
- **Process statements** MUST **include** the **input(s)** and **output(s)** that are used in each specified process.
- **All processes MUST be listed and written in the correct order**.

Control Structures

- **Control structures** are the way to **specify flow** in a **computer program**.
- There are **three (3)** main types of **control structures**:

Sequence

Series of steps executed line-by-line

Selection

Allows choices with multiple paths

Repetition

Repeats (loops) processes multiple times

Sequence Control Structure

Sequence control structure is a **series of actions** that are **sequentially executed** in the order they are written in an **algorithm**.

- There are **no conditions.**
- Performs actions **one after another** in a sequence.
- All actions will be executed, none will be skipped.
- Process keywords: **calculate, compute, count**

Sequence Example

A program accepts a price and discount rate from user and prints the price after discount.

Input: price, discount rate

Process: **calculate** price after discount **based on** price and discount rate

Output: price after discount

- basic formula for calculation processes: **calculate OUTPUT based on INPUT(s)**

Sequence Example

A program that prompts user to enter the total number of umbrellas he or she wants to buy. The price of each umbrella is RM3.00. This program will calculate and display the total price of these umbrellas to the user.

Input: number of umbrellas

Process: **calculate** total price of umbrellas **based on** the number of umbrellas and the price.

Output: total price of umbrellas

Selection Control Structure

Selection control structure performs actions based on a **certain conditions**.

- Outputs from selection control structures depends on conditions.
- A **condition** is like a **rule** that **decides** the output.
- There may be two or more outputs.
- Process Keywords: **if, else, otherwise, or, based on, depends, determine, find, check, compare.**

Selection Example

Print “The number is positive” if the number entered is greater than zero.

Input: number

Process: determine if the number is greater than zero

Output: “The number is positive” or no output (null)

- must consider **ALL possible outcomes** for the output.
  - If user enters positive number → “The number is positive”
  - If user enters negative number → no output (null)

Selection Example

A program calculates the discount to give to shoppers based on their total expenditure. Shoppers who spend between RM50 to RM100 will receive a 10% discount whereas shoppers who spend more than RM100 will receive a 15% discount.

Input: total expenditure

Process: determine discount based on total expenditure, calculate total expenditure after discount based on total expenditure and discount

Output: total expenditure after discount

What is Repetition Control Structure?

Repetition control structure runs the same statement more than once.

- It is used when there are many inputs, outputs, or processes to be repeated.
- Keyword: **repeat, loop, until, while, as long as, among.**
- Types of repetition loops:
  - Counter-controlled loop: number of repetitions is known
  - Sentinel-controlled loop: number of repetitions is unknown

Counter-controlled Loop Example

Print the numbers from 1 to 10.

Input: (none)
Process: loop through from 1 to 10 and print each number
Output: 1 2 3 4 5 6 7 8 9 10

- this is an example of a program that does the same thing each time and does not take any user input.
- the output is written as *values* since we know exactly what will be displayed.

Counter-controlled Loop Example

Write a program that reads the ages of 10 people, then displays the average age.

Input: age

Process: repeat input age 10 times and calculate the sum of the ages.
calculate the average based on sum

Output: average

- when we need to carry out some same process multiple times, we can repeat this process.
- here we needed to read 10 ages, so we repeat this reading of the age at the prosess.
- the input remains a singular descriptive name.

Counter-controlled Loop Example

A program asks teachers for *num_students,* which is the number of students in a class, then teachers can key in each students' exam mark. After all students' exam marks are entered, the program displays the highest mark and lowest mark.

Input: num_students, exam marks

Process: repeat input exam marks until num_students reached, determine the highest and lowest marks based on all the students' exam marks

Output: highest mark and lowest mark

Sentinel-controlled Loop Example

This program accepts a few marks as input. Once the user enters a negative value, the program will calculate and display the average marks of all the marks previously entered.

Input: marks

Process: repeat input marks until negative value entered,
calculate average marks based on marks entered

Output: average marks

- we do not know exactly how many marks will be entered, but the program will stop receiving input when a negative mark is entered by the user.

Sentinel-controlled Loop Example

Marina wants to write a program that displays whether or not a number entered is a multiple of 2. She wants the program to keep running until the input value -1 is entered.

Input: number

Process: determine if number entered is multiple of two; repeat input number until -1 entered

Output: “multiple of 2" or “not multiple of 2” messages

- we do not know exactly how many marks will be entered, but the program will stop receiving input when a negative mark is entered by the user.